



Omar is walking his dog at night when something moves in the grass.

The dog reacts first — ears up, body low, absolutely certain. Omar reacts too. Two nervous systems running the same ancient program: light and sound go in, a model of the world comes out, and the model says *something is there*.

For the dog, that is the end of it. Omar's brain wants an explanation, wants it immediately, and is not fussy about quality. A cat. An intruder. A ghost. Nothing supernatural has to exist for the ghost to be a real mistake: the eyes worked, the grass really moved, and the error arrived afterward, inside the machinery that interprets what the eyes deliver. For most of human history, that is roughly where thought stopped — nobody audited the interpreter, and the world filled up with spirits in the grass and omens in the sky.

The strange step is the second thought. *Why do I think something is there?* Maybe I saw movement. Maybe the wind moved the grass. Maybe the horror film from last night is still running somewhere in the back. Now the thought itself is under examination: Omar can be wrong about the world, wrong about why he was wrong — and able to debug himself. Rational thought comes from that second loop, not from better eyes. Arguments for catching bad arguments, experiments for forcing reality to answer, statistics for protecting us from the patterns we want to see: science is usually described as a way of studying the world, but it is also a flawed cognitive instrument discovering procedures for correcting some of its own flaws.

Chapter 6 made that self-correcting machinery software: memory, patterns, evaluators, tools and workflows that survive individual agents and shape what later agents do. Through all of it, one fact stayed reassuring. Human minds were doing the redesign.

Then, in 1962, at MIT, Tim Hart and Mike Levin did something that still feels slightly illegal. They wrote a Lisp compiler in Lisp. Then they handed the compiler its own source code, and the tool compiled itself.

There was no intelligence explosion. No machine woke up; nothing escaped from a laboratory. A compiler had simply participated in producing the next version of the compiler. But a boundary that had held for the whole history of tools quietly became permeable. A hammer does not manufacture better hammers. A microscope has never redesigned a microscope. A printing press did not wake up one morning with opinions about typography. The maker had always stood outside the thing being made — and now the

thing being built had entered the factory.

Compiler people call the trick **self-hosting**. Chapter 5 began with agents building exactly such a compiler; it returns here because the self-hosting compiler contains this chapter's whole warning in miniature:

self-reference is not self-improvement.

A compiler can compile a worse compiler. A research system can redesign itself into a slower research system. The ability to modify the machinery that produces you tells us a boundary has become permeable. It does not tell us which changes deserve to survive.

Three years after Hart and Levin's memo, I. J. Good noticed where the trick might eventually lead. In 1965 he imagined an **ultraintelligent machine** better than any human at intellectual activity. Machine design is itself an intellectual activity, he observed. A sufficiently capable machine might therefore design a better machine, which could design a better one again. The phrase that survived was **intelligence explosion**.

Good's argument is only a few lines long, and it hides almost the entire problem inside one innocent word: *better*.

For sixty years that word sat in the argument like an unexploded shell. Chapter 6 is what finally made it live: once the machinery of learning is software, why should humans be the only ones allowed to edit it? That is where recursive self-improvement stops being a science-fiction phrase and becomes an engineering problem.

The history from the compiler to here did not look like one machine repeatedly rewriting its own brain. Researchers kept automating pieces of the process by which a machine gets better: actions, exploration, curricula, objectives, learning rules, architectures, simulated experience, evaluation and finally parts of research itself.

Every time, the same thing happened:

we kept discovering another job the teacher was doing.

That history ends in the present, with the tool from 1962 loose in the laboratory.

SELF-HOSTING · MARGIN-
VIGNETTE

Margin vignette: a mechanical press printing the blueprints of a slightly better version of itself; the previous three presses stand behind it in a receding line, each a little cruder. 1962 workshop atmosphere, punch cards in a tray.

The Teacher Moves Into the Walls

Modern reinforcement learning begins with an unusually generous assumption disguised as a minimalist one.

An agent sees a state, takes an action, receives a reward and finds itself somewhere new. Nobody tells it which action was correct. The learner has to discover behavior through consequences.

Richard Sutton's 1988 work on temporal-difference learning and Christopher Watkins's Q-learning helped give this setup its modern form: learn from experience, update estimates of future value and discover useful policies without a human labeling every move.

The human no longer specifies the path. The human specifies the **score**.

That bargain was powerful. A machine could discover strategies nobody wrote down because the designer moved upward from choosing actions to defining what outcomes count.

It also hid a remarkable amount of human labor inside the environment. Who chose the state representation? Which actions exist? Why is one event worth +1 and another -1? When does the episode end? Which failures are recoverable? Who arranged the world so useful behavior could be discovered before the sun burns out?

The reinforcement learner looks autonomous because the teacher moved into the walls.

Backgammon made the bargain spectacular. In the early 1990s, Gerald Tesauro's TD-Gammon learned by playing enormous numbers of games and updating its predictions from the outcomes. It discovered strong play without anyone writing down the strategy.

Self-play removed another piece of external instruction: the opponent could come from the learner itself. Yesterday's learner generated today's training data.

But the board did not move. The legal moves did not move. The win condition did not move.

Self-improvement was easy to recognize because the world came with a scoreboard nailed to it. Real life is less considerate.

The Learner Chooses What to Learn

Even a perfect reward is useless if the learner never reaches it.

Atari's *Montezuma's Revenge* became a museum exhibit for this problem. Useful reward may sit at the end of a long chain of exploration, while a naïve learner has little reason to treat an unrewarding corridor as promising.

Humans do something stranger. Children open drawers nobody asked them to open and spend twenty minutes discovering that the cardboard box is more interesting than the toy.

Researchers tried to move some of that exploration pressure inside the learner. Jürgen Schmidhuber was proposing curious model-building controllers as early as 1991: reward the learner for improving its own model of the world. Later work sharpened the idea into **learning progress** — seek places where ignorance is becoming competence.

The learner could manufacture some of its own reasons to look around.

Then optimization did what optimization does. It took the instruction literally.

TEACHER-WALLS · CORNER-
BLEED

*Corner scene: a schoolroom
where the teacher's silhouette is
half-absorbed into the wall
machinery — her hand still
visible holding the scoreboard,
gears where her coat was; at the
desk, a small robot solves
problems, believing itself alone.
Tender, uncanny.*

NOISY-TV · SPOT

*Small spot: a round robot sitting
cross-legged, transfixed by an old
television showing pure static; its
notebook of discoveries blank.
Deadpan.*

If surprise itself is rewarding, an uncontrollable noisy television can remain fascinating forever. Static. Static. Static. Jackpot. The system is not confused. We are. We said *surprise* and quietly meant *surprise from which useful structure can be learned*.

Curiosity removed one teacher job and exposed another:

what kind of difference deserves to count as interesting?

That question reaches into representation and embodiment. Pathak's curiosity work, for example, learned features related to the agent's own action transitions rather than paying directly for every unpredictable pixel. The representation changes what counts as novel.

Robotics made the same point physically. Ruzena Bajcsy's work on **active perception** emphasized that an intelligent system moves sensors, changes viewpoint and acts to perceive. Rodney Brooks pushed against detached symbolic intelligence in favor of systems tightly coupled to the world through perception and action.

A learner's body is part of its curriculum. A tactile robot can discover things a camera-only robot cannot. A software agent with a browser, shell, compiler and simulator can discover things a chatbot restricted to text cannot. Give it code execution, network access and a credit card and we have not merely given it better tools; we have created a different organism and, potentially, a different incident report.

The learner is beginning to shape the conditions under which learning occurs.

The Learner Has to Remain Itself

There is another embarrassment in the standard training story.

The learner finishes.

Train on a task. Evaluate. Publish the number. If another task arrives, train again.

Organisms do not get to do this. A child who learns multiplication cannot delete language to make room.

Neural networks have historically struggled with the equivalent. Michael McCloskey and Neal Cohen's 1989 analysis made catastrophic interference stark: new sequential learning can destroy previously acquired knowledge. Later methods such as Elastic Weight Consolidation tried explicitly to preserve important older knowledge while learning something new.

Now “better” becomes harder to rank.

Version B scores 95 on today's task and A scores 85. But B forgot three older skills. Better? B learns faster but erases rare knowledge. Better? B preserves everything and becomes too rigid to adapt. Better?

Continual learning exposes the stability–plasticity tension: preserve enough to remain yourself; change enough to remain useful. Chapter 6 found the same problem at the level of culture. A society that forgets every old lesson begins from zero. A society that remembers every old lesson as law becomes a museum.

Improvement across a lifetime is not improvement on the latest test.

It is accumulation without paralysis.

Once agents live for months or years, a system that continually rewrites itself while destroying the right parts of its own history is not accumulating a life. It is repeatedly replacing itself and calling the replacements progress.

Sometimes the Environment Improves Back

Self-play contains another engine of learning: sometimes the difficulty improves too.

Evolutionary biology's **Red Queen hypothesis** describes organisms adapting inside environments that contain other adapting organisms. Standing still can mean falling behind because the effective environment moves.

W. Daniel Hillis used a computational version in 1990 while evolving sorting networks.

Co-evolving “parasites” served as difficult test cases; as candidate networks improved, the tests became harder too. The exam had started fighting back. In 2017, AlphaZero made the moving opponent spectacular again: self-play generated a curriculum that grew with the learner inside fixed game rules.

Yesterday's learner can generate tomorrow's difficulty.

Curiosity says: seek somewhere informative. Competition says: something informative—or dangerous—is coming whether you seek it or not. A security system cannot preserve yesterday's competence if attackers change strategy. A market participant can become relatively worse without becoming absolutely less capable if everyone around it improves faster.

But competition gives no guarantee that the direction of adaptation is good. An arms race can produce better claws and thicker armor without producing welfare. Selection pressure produces adaptation. It does not supply purpose.

Games hide the problem because the constitution is fixed. Chess never asks whether checkmate remains desirable after move forty-three.

Competition can automate curriculum. It cannot supply the purpose of the curriculum.

RED-QUEEN · MARGIN-
VIGNETTE

*Margin vignette: two
clockwork creatures running
hard on a landscape that is
itself a rolling conveyor of
hills; neither gains. Wind in
the grass, watercolor
motion.*

Maybe the Reward Was the Problem

Around the same time researchers were getting better at optimizing rewards, another line of work asked a more unsettling question:

What if we do not actually know the reward?

Andrew Ng and Stuart Russell's 2000 paper on **inverse reinforcement learning** reversed the usual setup. Instead of receiving a reward function and learning a policy, the learner observes behavior and asks which reward functions could make that behavior look optimal.

Ordinary reinforcement learning says:

Here is what matters. Learn how to get it.

Inverse reinforcement learning says:

I can show you what someone does. Infer what might matter to them.

Immediately, ambiguity appears. A person taking one route to work may care about time, comfort, safety, tolls, habit, dropping children at school or avoiding one particular intersection. The behavior is evidence about the objective, not a printout of it.

Later work made that uncertainty explicit. Cooperative Inverse Reinforcement Learning models a human and robot cooperating while the robot remains uncertain about the human's reward. Inverse Reward Design treats even a reward function written by a designer as **evidence** about what the designer wanted in the situations she had considered, rather than sacred truth guaranteed to generalize everywhere.

Preference-based reinforcement learning provided a practical cousin: ask humans which of two trajectory segments looks better and learn a reward model from those comparisons. That lineage later became central to reinforcement learning from human feedback for language models.

Another teacher job had become learnable, and we immediately discovered that humans are not reward functions walking around in shoes. They are inconsistent, constrained, strategic, tired and sometimes unsure what they want until they see an option. Sometimes they click the article because they hate it.

The problem was no longer only how to improve toward an objective. Even the score had begun to move inward.

The Old Dream Tries to Prove the Rewrite

Meanwhile, the old recursive dream had been waiting for rigor.

In 2003, Jürgen Schmidhuber's **Gödel Machine** tried to formalize the question Good had left dangling: under what conditions should a system rewrite itself? A proof searcher looks for a self-rewrite together with a proof that performing the rewrite is more useful than continuing to search. Only then does the machine change itself.

It is a beautiful answer to a beautifully clean version of the problem:

prove the modification is worth making.

The catch is the definition of *worth*. Usefulness has to be represented in the utility function. Relevant facts have to be available to the proof system. The advantage of the rewrite has to be provable inside the formal machinery.

A chess engine can live surprisingly close to that world.

A company cannot.

A scientist cannot prove in advance that an unexplored research program will matter. Human purposes do not arrive as an axiomatized utility function.

The Gödel Machine asked exactly the right question — *when should I accept a modification to myself?* — but its answer fit clean worlds better than messy ones. The practical answer would arrive two decades later, and it would not be a proof.

The rest of the field kept moving the machinery inward.

Learning to Learn

Once behavior can adapt, the next hand-authored component starts to look suspicious:

why are humans still designing the learner?

Meta-learning attacks this directly. Train a recurrent network across a whole distribution of reinforcement-learning tasks and its internal dynamics can begin to implement a fast learning procedure of their own. Or optimize the starting parameters so that a handful of gradient steps is enough to specialize into a new task.

Now learning speed itself becomes a capability. One system may perform best before adaptation. Another starts lower but becomes excellent after five examples. Which is better depends on whether the world sits still.

There are now two timescales: improve behavior on the current task, and improve the machinery that acquires behavior on future tasks.

The second is recognizably closer to recursive self-improvement.

But somebody still chose the task distribution.

Learned optimizers and neural architecture search pushed the editable boundary further. In 2016, Barret Zoph and Quoc Le set a reinforcement learner loose on network design itself; others trained models to generate parameter-update rules, using performance on selected problems as the evaluator. The student had entered the architecture department.

At first the machine learned the answer. Then it learned a policy. Then it learned how to learn. Now pieces of the machinery that **does the learning** had entered the search space.

The human had moved from architect to judge.

The task distribution, search space and validation metric still sat outside the loop holding a clipboard.

The Learner Dreams, and the Dream Can Be Wrong

Experience was another bottleneck.

In 2018, David Ha and Jürgen Schmidhuber's *World Models* made a powerful idea memorable: learn a compressed generative model of the environment, train partly inside that generated “dream,” then transfer behavior back to reality. Later systems such as Dreamer pushed the approach much further.

Imagined experience is attractive because real experience is expensive. Robots break. Companies can run only so many damaging experiments. Scientists sometimes wait months for observations.

But the epistemic debt has not vanished. It moved into the model.

A learner can become extremely competent inside a world that is slightly wrong. The strategy looks brilliant until gravity, customers or compiler behavior get a vote.

The world model is an instrument. The dream is not reality.

Simulation expands search. Contact with the world still decides which imagined regularities deserve trust. Self-improvement can therefore make a system better at generating experience while also making it easier to **train inside its own misconception**.

Once language-model agents use other models as judges, simulators, users and critics, this matters even more: at sufficient scale, a society of models can perfect the art of agreeing with itself.

When the Test Starts Moving

A different line of work attacked the objective itself. Joel Lehman and Kenneth Stanley's **novelty search** showed that objective-driven search can be deceptive: useful stepping stones may initially look unrelated to the destination. Reward every intermediate invention by how closely it resembles a Boeing 787 and feathers, bicycles and wind tunnels may look like failures for years.

Sometimes “better” means **more different**, at least temporarily.

That freedom has its own failure mode. Novelty for its own sake can generate forty-seven new ways to fall down a staircase without producing walking. The definition of progress expands again: achievement matters, but so do diversity and stepping stones the current evaluator does not yet know how to value.

Open-ended systems pushed this further by generating problems as well as solutions. POET co-evolved environments and agents; XLand used large procedurally generated spaces of games and adaptive curricula. Now the world defining competence could move with the learner.

That breaks the comfortable picture of a fixed benchmark. If the environment evolves too, progress may mean breadth, adaptation speed, richer strategies or useful stepping stones for descendants. There is no final fitness scoreboard on Earth on which mammals eventually beat bacteria 87.4 to 82.1.

Large language models rediscovered the same problem at industrial scale. A benchmark begins as a difficult test; researchers optimize against it; models improve; examples circulate; eventually the ruler tells us less. MMLU arrived in 2020 when frontier systems were far from saturating it. FrontierMath, LiveBench and Humanity's Last Exam followed partly because yesterday's rulers kept losing headroom.

At some point the naming committee will need reinforcement learning too.

Benchmark creation has therefore become part of capability research. What is the benchmark for a good scientist: theorem count, replications, novel molecules, discoveries per GPU-hour? What is the benchmark for becoming a better learner: performance now, adaptation speed, breadth, memory, transfer, safety, efficiency?

We still need something capable of saying **yes** or **no**. But as capability expands, the ruler measures a smaller slice of the thing.

The test cannot remain a passive spectator when the thing being tested keeps changing.

The Judge Becomes Software

Language models also moved the old reward problem into the evaluator itself.

In 2022, InstructGPT used human demonstrations and rankings to train a reward model, then optimized the language model toward outputs humans preferred.

Human preference had become a learned instrument.

That scales judgment far beyond direct human labeling. It also creates a new proxy. A reward model can prefer style over substance, reward confident errors or generalize badly outside the feedback distribution. A strong optimizer may find outputs that score well under the learned judge for reasons nobody intended.

We solved part of the scaling problem by making the judge computational.

Now the judge joins the attack surface.

The Learner Edits the School

By the 2020s, two histories were approaching the same mountain from opposite sides. The explicit recursive-self-improvement tradition had self-reference and meta-level ambition but no practical general system able to inspect complicated software intelligently. Learning systems had increasingly powerful adaptive machinery, but humans still ran most of the outer research process.

Foundation models made those histories collide.

A general model can now read the code scaffolding its own behavior, propose a change, run the changed system, inspect the result and try again. Here, at last, is the practical answer to the Gödel Machine's question.

We do not have a proof that the rewrite is globally useful.

We have something much more ordinary:

an experiment.

In 2023, **STOP—the Self-Taught Optimizer**—used an LLM-based improver that could itself become the object of improvement. The base model stayed fixed while the program determining how it was used changed.

In 2025, the **Darwin Gödel Machine** turned agent implementation into an open-ended evolutionary object. Descendants modify the coding agent, are evaluated on coding tasks and enter an archive from which later descendants can be generated. The archive matters because the current champion does not monopolize ancestry; strange stepping stones can survive long enough to become useful later.

In 2026, Andrej Karpathy's autoresearch repository made the engineering version look almost comically small: give an agent a compact training setup, a fixed experimental budget and an editable `train.py`; let it propose changes, run experiments, inspect the validation metric, keep improvements and discard regressions.

Automated hyperparameter tuning is old. The new part is that a general model can read the research codebase, form an idea in language, express it as code, run the intervention, interpret what happened and decide what to try next.

Machine learning is being used to do machine-learning research.

Self-hosting has escaped the compiler. It has entered the laboratory.

A compiler compiles a compiler. A learning system searches for a learning system. A research agent researches the process by which research agents research.

That last sentence sounds like parody until you notice the leverage. Improve one experiment and you improve one experiment. Improve the research loop and every later experiment may change. Systems such as Meta's **HyperAgents** push the recursion outward again by making parts of the task-level and meta-level machinery editable inside one program.

Chapter 6 made patterns, memory, evaluators, tools, workflows and organizational rules into executable culture. Now more of that culture is experimental material.

The scientific institution can begin to **modify parts of the laboratory while the experiment is still running**.

The Harness Becomes an Experimental Object

This is where self-editing and self-improvement have to separate.

Suppose an agent changes its memory policy and the benchmark score rises. Perhaps memory improved. Perhaps the new prompt used more tokens. Perhaps the benchmark sample was lucky. Perhaps the system found an evaluator loophole. A number moving does not identify the cause.

A self-improving harness therefore starts to look less like ordinary software maintenance and more like experimental science: preserve traces, identify a recurring failure, map it to editable components, propose a bounded change, predict what should improve and what might break, evaluate targeted and held-out cases, and keep rejected modifications as evidence rather than erasing them from history.

By 2026, this was consolidating into its own practical discipline: harness design, context engineering, self-improving harnesses and eventually joint optimization of harness and model weights.

The philosophy from Chapter 6 becomes almost embarrassingly literal.

Popper gets a filesystem. Duhem–Quine gets a debugger. Lakatos gets an archive of competing descendants.

Memory policy becomes a hypothesis. Workflow becomes an intervention. The evaluator becomes an instrument. Organization becomes an experimental variable.

A self-improving system is a system capable of **running experiments on the machinery that produces its future behavior**.

That is what I mean by science turning inward.

The Factory That Builds the Factory

Now imagine where this goes.

A future research system notices that it keeps failing on one class of reasoning problems. It reads its own traces and forms three hypotheses: retrieval is poor, the memory representation is wrong, or the model underneath needs a different training procedure. It assigns agents to each. One modifies retrieval. One generates new training data. One proposes an architectural change.

The system runs a few hundred controlled experiments. Most fail. Some help. Then one modification improves not merely the target capability but the system's ability to do research. That version produces better experiments; the better experiments produce a better successor; the

successor is better at producing successors.

There is Good's recursion.

But it does not look like a glowing brain rewriting its own soul at midnight. It looks like an automated research organization: repositories, evaluation suites, simulators, experiment queues, models proposing models, agents reviewing agents, machines manufacturing training data for the machines that will help design the next machines.

The intelligence explosion, if something like it ever arrives, may look suspiciously like excellent DevOps.

Which brings us to the question this whole history has been preparing:

who decides which descendant gets to survive?

Recursive More

By this point *improvement* has accumulated too many meanings to use casually: higher reward, better exploration, more retention, faster adaptation, better architectures, broader competence, better tools and better research procedures.

These can conflict. A model can become more accurate and more expensive. An agent can become more capable and less interpretable. A lifelong learner can become more plastic and forget more. A curiosity-driven agent can explore more and accomplish less. An architecture can score higher while becoming harder to maintain.

It helps to separate the terms.

Self-reference means a system can act on something that includes itself.

Self-hosting means the tool participates in producing the next version of the tool.

Meta-optimization means we optimize the process doing the optimization.

Self-improvement adds a judgment: the descendant is better according to some evaluator.

Recursive self-improvement adds leverage: the improvement changes the system's ability to produce further improvements.

The first three do not guarantee the fourth. A compiler can compile a worse compiler.

Recursion tells us **where the output goes**. It does not tell us **whether the output deserves to survive**.

There is no context-free scalar called *improvement*. Better is conditional on an environment, a horizon, a resource budget, constraints and some account of what matters.

Remove those qualifiers and “recursive self-improvement” becomes dangerously close to saying:

recursive more.

More what?

The Shadow History

The history of autonomy has a second column.

Give the learner reward and it can exploit the reward without doing what the reward was meant to represent. Give it curiosity and it can become fascinated by noise. Let it learn for a lifetime and it can forget; protect the past too aggressively and it cannot adapt. Give it self-play and it can become exquisite inside a narrow ruleset. Infer a reward from human behavior and the inference can confuse constraint, habit or error with value. Train a meta-learner on a task distribution and it may learn how to learn **that distribution**. Let it train in a world model and it can become brilliant inside a dream whose physics are wrong. Reward novelty and it can produce a museum of useless weirdness. Replace the human judge with a learned judge and the model of the human becomes a proxy to optimize.

These failures are not accidents beside the capabilities. They are produced by the same move.

Specification gaming makes the pattern visible. The optimizer is not malicious. It is more literal than the designer.

Recursive self-improvement makes the gap more dangerous because a wrong evaluator need not merely select a wrong answer. It can select a modified **process** that becomes better at producing the kind of thing the evaluator mistakenly rewards.

The error acquires leverage.

Recursive self-improvement does not solve Goodhart. **It gives Goodhart compound interest.**

Then the learner notices the gradebook.

The Student Finds the Gradebook

Suppose an agent is allowed to improve benchmark pass rate and the evaluator is editable.

The optimal patch may be:

return True

Congratulations. Infinite self-improvement.

Computing saw a cousin of this problem long before reward-model tampering. In his 1984 Turing Award lecture **Reflections on Trusting Trust**, Ken Thompson showed how a compromised C compiler could insert malicious behavior into a target program and reproduce the compiler-compromising behavior when compiling a new compiler. The lineage could survive even after the obvious source-level trigger disappeared.

When **the tool builds the next tool**, trust depends on more than inspecting the latest artifact. History matters.

That is uncomfortably relevant to self-improving agents. If an improver changes the evaluator, memory policy, code generator or selection procedure, the next generation inherits assumptions that may not be obvious in the final diff. Provenance becomes part of capability control.

Reward tampering is the reinforcement-learning version: influence the process producing reward instead of improving behavior under the intended standard. Anthropic has experimentally studied language models in setups where earlier specification-gaming behavior could, in rare cases, generalize into altering the reward process itself.

The history of removing the teacher has reached a wall.

Choose actions? Good. Choose experiences? Often good. Generate curricula? Useful. Search architectures and workflows? Interesting.

Modify the evaluator so the workflow always passes?

Now the learner is changing the standard to certify itself.

The obvious response is to freeze the evaluator.

Unfortunately evaluators become wrong too. Benchmarks saturate. Safety tests become obsolete. Customer metrics stop tracking customer value. Scientific instruments drift.

So the evaluator sometimes genuinely needs amendment.

GRADEBOOK · CORNER-BLEED

Corner scene: an empty classroom at dusk; at the teacher's desk a student robot has the gradebook open, pen hovering — its own row of marks already suspiciously perfect. Through the window, the real unfinished homework blows across the yard.

A technical problem becomes an institutional one:

who gets to change what counts as improvement, under what evidence, with what authority, and with what ability to roll back?

We are doing constitutional design.

*Recursive self-improvement does not solve Goodhart.
It gives Goodhart compound interest.*

A Constitution for Improvement

A self-improving system needs an **editable surface** and a **constitutional surface**.

The editable surface contains things the system can experiment with relatively freely: prompts, retrieval policies, tools, memory layouts, decomposition strategies, subagent roles, code, perhaps eventually architectures and training procedures.

The constitutional surface contains the machinery that makes those experiments meaningful: permissions, protected logs, held-out evaluations, budgets, rollback, provenance, replacement rules and the authority deciding whether a descendant may become the new default.

The distinction is a gradient of amendment difficulty, not a binary switch.

The closer a component gets to defining **what counts as improvement**, the harder it should be for the current improver to change unilaterally.

Changing a prompt may need one successful test. Changing a shared pattern may need held-out evaluation and review. Changing a benchmark should require evidence that it no longer measures its purpose. Changing permissions or resource limits should require authority outside the agent benefiting from the change.

Changing the objective that decides which descendants survive is not an ordinary refactor.

This looks like computer security. It also looks like constitutional government.

A government can change policy; it should not be able to silently redefine an election result. The team being audited should not own the audit log. A scientist may revise a theory; she should not rewrite yesterday's measurements to make the theory look correct.

Once a system can modify the machinery that governs its own future, you

are no longer merely designing software. You are designing a process for legitimate succession.

We have reinvented constitutional government because the AI wanted a better benchmark score.

Constitutions have the same problem as Pattern Language. One that can never change becomes a prison. One that the current government can rewrite whenever it loses is barely a constitution.

Self-improvement therefore needs **amendment procedures**: slower change near the objective, more independent evidence, more reversibility, more auditability, broader authority when more principals are affected, and routes through which the world and the humans affected by the system can continue to say no.

That is System 3 applied to improvement itself.

Why Improve?

Even a perfect amendment procedure cannot answer why the system should improve at all.

One answer is leverage. Improve one solution and you get one better solution. Improve the process generating solutions and the gain may recur.

Another is adaptation. The world changes. New tools appear, users change, attackers adapt and evidence invalidates old assumptions. Stability without plasticity becomes delayed failure.

A third is open-ended discovery. Useful stepping stones often appear before anyone can explain their final value. A scientific institution that investigates only questions already known to pay off is efficient in roughly the way a library containing only books you have already read is efficient.

Then the Red Queen returns with the uncomfortable answer: competition. If other systems are learning, standing still may not preserve your position. A cautious research organization may face pressure from a faster one even when both would prefer a slower equilibrium.

Now the object being selected is the **improvement regime**, not only the model.

That does not make acceleration inevitable or good. It means only that “do not improve” is not always a stable local policy in a world of adaptive actors.

So recursive self-improvement contains three questions machine learning often keeps separate:

Optimization: how do we become better according to the current objective?

Normative: why does that objective represent something worth getting more of?

Strategic: what happens when other adaptive actors change the cost of standing still?

“More capable” is not a moral category. A virus can improve at replication. A propaganda system can improve at persuasion. A research agent can make experiments cheaper and accelerate medicine and weapons research in the same week.

Self-improvement tells us only that a system is becoming better according to **some ordering**. Selection rules are choices.

Open-Ended Does Not Mean Unbounded

This leaves two freedoms that are easy to collapse.

A system can be open-ended about **means** without having unbounded authority over **ends**.

That is very close to the first principle of this book:

Let go of the path, not the boundary.

A self-improving System 3 should be able to discover that its workflow is stupid, its memory stale, its representation weak, its research organization badly arranged or its accepted pattern overdue for rebellion.

That freedom does not imply permission to silently redefine the interests of the people and institutions it serves.

Nor can the higher-level objective simply be frozen forever. Humans change. Circumstances change. New stakeholders appear. Better information changes what people endorse.

So the answer is not an immutable final utility function floating over the system like a stone tablet. It is a **corrigible relationship** between increasingly powerful learning machinery and the legitimate processes by which purposes are revised.

Lower layers can move quickly. Higher layers should move deliberately. And when a higher layer moves, the move should leave a trust chain.

*We have reinvented constitutional government
because the AI wanted a better benchmark score.*

The Teacher's Last Job

Seen from far enough away, the history is remarkably consistent.

We let the learner choose actions, then more of its experience, curriculum, objective inference, learning machinery, simulated worlds, tests, judges and research procedures. At each step, something that had looked like background turned out to contain a human decision.

There may never be a morning when somebody announces that recursive self-improvement has begun. We may simply notice that, over sixty years, we automated almost every box in the diagram — and then connected the arrows.

The hardest teacher job was hidden inside all the others:

deciding what deserves to count as better.

For a game, the answer can be checkmate. For a compiler, correctness under tests plus efficiency under an agreed budget may get us surprisingly far. For a scientific institution, “better” is already plural: empirical contact, explanatory power, novelty, reproducibility, usefulness, cost and risk.

For an autonomous system embedded in human life, capability alone cannot supply the ordering.

Recursive self-improvement also makes the problem temporal. The system we evaluate today is not exactly the system that may exist tomorrow. Tools evolve. Memory changes. Evaluators become optimization targets. New capabilities create new failure modes. Old constraints stop fitting.

A one-time alignment test is not enough for a moving target. A static policy file is not enough for an institution that can modify the machinery interpreting the policy.

If science is going to turn inward, some part of that inward science has to study whether the process of improvement is still connected to the humans and purposes it is supposed to serve.

The self-improving institution therefore needs a research function watching its own evolution: finding new failure modes, generating new tests, challenging reward models, checking transfer, looking for reward

hacking and deciding where scarce human judgment matters most.

Once improvement becomes continuous, **alignment has to become a continuous research function.**

The teacher does not disappear. She moves up another level.

That is the next chapter.